

Chương 5: Thao tác với cơ sở dữ liệu trong Python

- Cài đặt thư viện MySQL Connector cho Python:
pip install mysql-connector-python
- Cài đặt MySQL

Kết nối MySQL

```
import mysql.connector
from mysql.connector import errorcode

config = {
    'user': 'root',
    'password': '123456',
    'host': 'localhost'
}

# Connect to MySQL
try:
    conn = mysql.connector.connect(**config)
    cursor = conn.cursor()
    print("Connect MySQL Server successfully")
except mysql.connector.Error as err:
    if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
        print("Connect error, please check username/password")
    else:
        print(err)
```

Tạo Cơ sở dữ liệu mới

```
# Create new DB
try:
    cursor.execute("CREATE DATABASE library_db")
    print("Create DB successfully")
except mysql.connector.Error as err:
    if err.errno == errorcode.ER_DB_CREATE_EXISTS:
        print("DB existing")
    else:
        print(err)
```

Tạo Table mới

```

config['database'] = 'library_db'
try:
    conn.database = config['database']
    print("Connect DB successfully")
except mysql.connector.Error as err:
    print("Can not connect DB")
    print(err)

sqlBook = (
    "CREATE TABLE Books ("
    "  book_id INT AUTO_INCREMENT PRIMARY KEY,"
    "  title VARCHAR(255) NOT NULL,"
    "  author VARCHAR(255),"
    "  published_date DATE"
    ")")

try:
    cursor.execute(sqlBook)
    print(f"Create Book table successfully")
except mysql.connector.Error as err:
    if err.errno == errorcode.ER_TABLE_EXISTS_ERROR:
        print(f"Book table existing")
    else:
        print(err)

```

Xóa Table

```

drop_table_query = "DROP TABLE IF EXISTS Books"

try:
    cursor.execute(drop_table_query)
    print("Delete table successfully")
except mysql.connector.Error as err:
    print(f"Error: {err}")

```

Insert data vào Table

```

sqlInsertBook1 = ("INSERT INTO Books "
                  "(title, author, published_date) "
                  "VALUES ('Programming Python', 'Raymond', '2023-01-01')")

# Excute Insert
try:
    cursor.execute(sqlInsertBook1)
    conn.commit()
    print("Insert Data successfullly")
except mysql.connector.Error as err:
    print(f"Error: {err}")
    conn.rollback()

```

Câu lệnh update

```

#Excute UPDATE
sql_book_update = ("UPDATE Books "
                   "SET title = %s "
                   "WHERE book_id = %s")

update_data = ('Advanced Python', 1)
try:
    cursor.execute(sql_book_update, update_data)
    conn.commit()
    print("Update data successfully")
except mysql.connector.Error as err:
    print(f"Error: {err}")
    conn.rollback()

```

Câu lệnh delete

```
sql_delete = "DELETE FROM Books WHERE book_id = %s"

book_id_to_delete = (1,) # Delete record which ID = 1

# Excute DELETE
try:
    cursor.execute(sql_delete, book_id_to_delete)
    conn.commit()
    print("Delete successfullly")
except mysql.connector.Error as err:
    print(f"Error: {err}")
    conn.rollback()
```

Đóng kết nối

```
cursor.close()
```

```
conn.close
```

Chương 6: Lập trình đa luồng trong Python

Một luồng (thread) là một đơn vị cơ bản trong CPU, và nó chia sẻ dữ liệu với các luồng khác trong cùng một tiến trình (process).

Tiến trình (process) là một đơn vị hoạt động của một ứng dụng, chứa thông tin về tài nguyên và trạng thái thực thi của chương trình.

Một điểm khác biệt quan trọng là nhiều luồng (threads) trong cùng một tiến trình (process) chia sẻ cùng một không gian bộ nhớ, trong khi các tiến trình khác nhau không chia sẻ bộ nhớ với nhau. Điều này cho phép các luồng trong cùng một tiến trình đọc và ghi vào cùng một cấu trúc dữ liệu, cũng như giao tiếp dễ dàng với nhau.

Đa luồng (multithreading) là một kỹ thuật trong lập trình cho phép một chương trình thực thi nhiều luồng (threads) đồng thời. Mỗi luồng là một đơn vị thực thi độc lập trong một tiến trình (process), và các luồng trong cùng một tiến trình chia sẻ cùng một không gian bộ nhớ.

Đơn luồng: Chương trình đơn luồng chỉ có một luồng thực thi tại một thời điểm. Đây là mô hình đơn giản nhất, nơi tất cả các tác vụ được thực hiện tuần tự.

```
import time

class Task:
    def __init__(self, name):
        self.name = name

    def run(self):
        for i in range(5):
            print(f"{self.name} - {i}")
            time.sleep(1)

def main():
    task1 = Task("Task 1")
    task2 = Task("Task 2")

    task1.run()
    task2.run()

if __name__ == "__main__":
    main()
```

```
Task 1 - 0
Task 1 - 1
Task 1 - 2
Task 1 - 3
Task 1 - 4
Task 2 - 0
Task 2 - 1
Task 2 - 2
Task 2 - 3
Task 2 - 4
```

Threading Module có một số phương thức sau đây:

- **threading.activeCount():** Trả về số đối tượng thread mà là active.
- **threading.currentThread():** Trả về số đối tượng thread trong Thread control của Caller.
- **threading.enumerate():** Trả về một danh sách tất cả đối tượng thread mà hiện tại là active.
- Bên cạnh đó, threading Module có lớp Thread để triển khai đa luồng. Lớp này có các phương thức sau:
 - **run():** Là entry point cho một Thread.
 - **start():** Bắt đầu một thread bởi gọi phương thức run().
 - **join([time]):** Đợi cho các thread kết thúc.

- **isAlive():** Kiểm tra xem một thread có đang thực thi hay không.
- **getName():** Trả về tên của một thread.
- **setName():** Thiết lập tên của một thread.

Một số vấn đề trong multithread

- **Race:** Xảy ra khi hai hoặc nhiều thread cùng truy cập và sửa đổi cùng một dữ liệu mà không có cơ chế đồng bộ hóa, dẫn đến tình trạng dữ liệu không chính xác.
- **Livelock:** Xảy ra khi các thread tiếp tục thay đổi trạng thái của chúng để cố gắng giải quyết tình trạng xung đột, nhưng không bao giờ thực hiện được công việc vì các thread luôn thay đổi trạng thái của nó.

Đồng bộ hóa các Thread trong Python

Trong Python, module `threading` cung cấp các công cụ để đồng bộ hóa các luồng (threads), và một trong những công cụ quan trọng là **Lock**.

Lock và Các Phương Thức Của Nó

Tạo Một Lock

Để tạo một đối tượng lock mới, bạn sử dụng phương thức `Lock()` từ module `threading`:

```
import threading

lock = threading.Lock()
```

❖ Phương Thức `acquire()`

Phương thức `acquire()` được sử dụng để lấy lock. Nó có hai tùy chọn quan trọng:

- **blocking=True (mặc định):** Luồng sẽ chờ cho đến khi lock được giải phóng nếu lock đang bị chiếm giữ. Nếu lock đã có người giữ, luồng sẽ đợi cho đến khi lock trở nên sẵn sàng.
- **blocking=False:** Luồng sẽ không chờ mà ngay lập tức trả về kết quả. Nếu lock có sẵn, nó sẽ lấy lock và trả về `True`. Nếu lock đang bị chiếm giữ, nó sẽ trả về `False` ngay lập tức.

Phương Thức `release()`

Phương thức `release()` được sử dụng để giải phóng lock khi luồng không còn cần đến nó nữa. Điều này cho phép các luồng khác có thể lấy lock và tiếp tục

thực thi.

```
import threading
import time

lock = threading.Lock()

def thread_function(name):
    print(f"Thread {name} is getting lock")
    acquired = lock.acquire(blocking=False)
    if acquired:
        try:
            print(f"Thread {name} get lock successfully.")
            time.sleep(2)
        finally:
            print(f"Thread {name} release.")
            lock.release()
    else:
        print(f"Thread {name} can not get lock")

def main():
    threads = []
    for i in range(3):
        t = threading.Thread(target=thread_function, args=(i,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

if __name__ == "__main__":
    main()
```